Weather Detection (Weather Type Classifier)

Predicts the weather type (**sun, rain, drizzle, fog, snow**) from precipitation, temperature, wind and month.

Dataset: Seattle weather, 2012 to 2015 (1,461 days). Run the cells from top to bottom (Runtime → Run all) and upload `weather.csv` when asked.

1. Imports

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (ConfusionMatrixDisplay, accuracy_score,
                             classification_report, f1_score)
from sklearn.model_selection import StratifiedKFold, cross_val_score, train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
```

2. Upload dataset

Select `weather.csv`.

```
from google.colab import files
uploaded = files.upload()
```

Choose Files weather.csv

weather.csv(text/csv) - 48219 bytes, last modified: 10/22/2026 - 100% done

```
Saving weather.csv to weather.csv
```

3. Load and explore the data

```
df = pd.read_csv("weather.csv").dropna().drop_duplicates()
df["date"] = pd.to_datetime(df["date"])
df["month"] = df["date"].dt.month

print("Dataset shape:", df.shape)
print(df["weather"].value_counts())
df.head()
```

```
Dataset shape: (1461, 7)
weather
rain       641
sun        640
fog        101
drizzle     53
snow        26
Name: count, dtype: int64
```

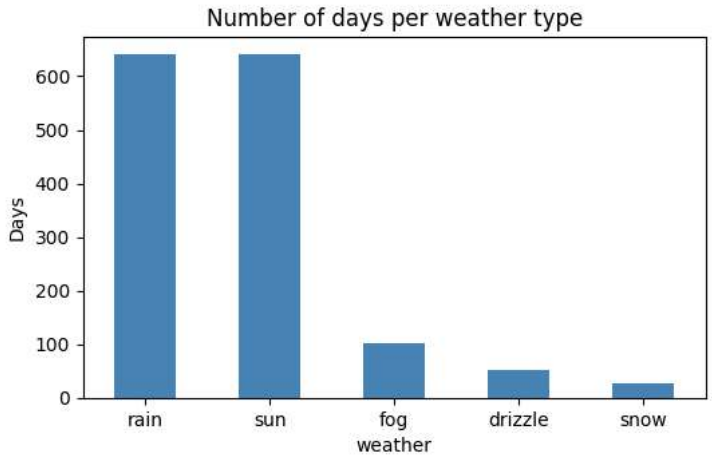
	date	precipitation	temp_max	temp_min	wind	weather	month
0	2012-01-01	0.0	12.8	5.0	4.7	drizzle	1
1	2012-01-02	10.9	10.6	2.8	4.5	rain	1
2	2012-01-03	0.8	11.7	7.2	2.3	rain	1
3	2012-01-04	20.3	12.2	5.6	4.7	rain	1
4	2012-01-05	1.3	8.9	2.8	6.1	rain	1

Next steps: Generate code with df | New interactive sheet

```
# Class distribution and average values for each weather type
fig, ax = plt.subplots(figsize=(6, 3.5))
```

```
df["weather"].value_counts().plot(kind="bar", color="steelblue", ax=ax)
ax.set_title("Number of days per weather type")
ax.set_ylabel("Days")
plt.xticks(rotation=0)
plt.show()

df.groupby("weather")[["precipitation", "temp_max", "temp_min", "wind"]].mean().round(2)
```



weather	precipitation	temp_max	temp_min	wind
drizzle	0.00	15.93	7.11	2.37
fog	0.00	16.76	7.98	2.48
rain	6.56	13.45	7.59	3.67
snow	8.55	5.57	0.15	4.41
sun	0.00	19.86	9.34	2.96

4. Feature engineering

Three extra features are created from the existing columns.

```
df["temp_range"] = df["temp_max"] - df["temp_min"]
df["temp_avg"] = (df["temp_max"] + df["temp_min"]) / 2
df["wet"] = (df["precipitation"] > 0).astype(int)

FEATURES = ["precipitation", "temp_max", "temp_min", "wind", "month",
            "temp_range", "temp_avg", "wet"]
X, y = df[FEATURES], df["weather"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

```
Train: (1168, 8)  Test: (293, 8)
```

5. Compare models (5-fold cross-validation on training data)

The classes are imbalanced, so **macro F1** is used to pick the best model.

```
models = {
    "Logistic Regression": Pipeline([("sc", StandardScaler()),
                                     ("m", LogisticRegression(max_iter=1000, class_weight="balanced"))]),
    "KNN": Pipeline([("sc", StandardScaler()), ("m", KNeighborsClassifier(n_neighbors=7))]),
    "Decision Tree": DecisionTreeClassifier(max_depth=6, class_weight="balanced", random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=300, min_samples_leaf=3,
                                            class_weight="balanced_subsample", random_state=42),
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
rows = []
```

```
for name, m in models.items():
    acc = cross_val_score(m, X_train, y_train, cv=cv, scoring="accuracy").mean()
    f1 = cross_val_score(m, X_train, y_train, cv=cv, scoring="f1_macro").mean()
    rows.append((name, acc, f1))

results = pd.DataFrame(rows, columns=["Model", "CV Accuracy", "CV Macro F1"])
results = results.sort_values("CV Macro F1", ascending=False).round(4)
results
```

	Model	CV Accuracy	CV Macro F1
3	Random Forest	0.8296	0.5727
1	KNN	0.8451	0.5119
0	Logistic Regression	0.6789	0.4887
2	Decision Tree	0.6773	0.4730

Next steps: Generate code with `results` New interactive sheet

6. Train the best model and test it

```
best_name = results.iloc[0]["Model"]
best = models[best_name].fit(X_train, y_train)
pred = best.predict(X_test)

print("Best model:", best_name)
print(f"Test accuracy: {accuracy_score(y_test, pred):.4f}")
print(f"Test macro F1: {f1_score(y_test, pred, average='macro'):.4f}\n")
print(classification_report(y_test, pred, zero_division=0))
```

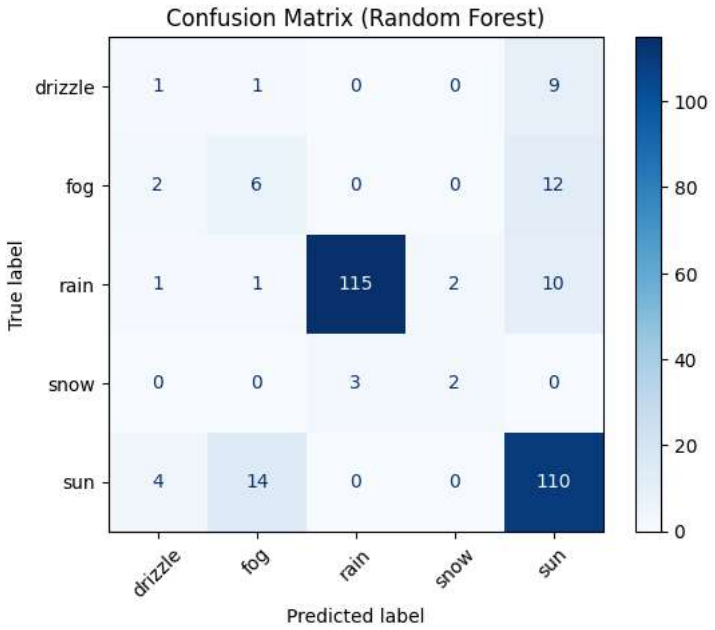
```
Best model: Random Forest
Test accuracy: 0.7986
Test macro F1: 0.5169

              precision    recall  f1-score   support

     drizzle       0.12      0.09      0.11        11
         fog       0.27      0.30      0.29        20
        rain       0.97      0.89      0.93       129
        snow       0.50      0.40      0.44         5
         sun       0.78      0.86      0.82       128

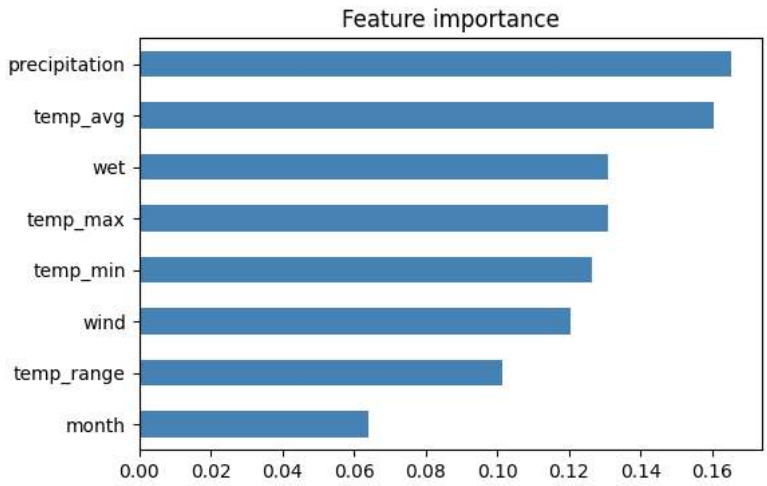
    accuracy                           0.80       293
   macro avg       0.53      0.51      0.52       293
weighted avg       0.80      0.80      0.80       293
```

```
ConfusionMatrixDisplay.from_predictions(y_test, pred, cmap="Blues", xticks_rotation=45)
plt.title(f"Confusion Matrix ({best_name})")
plt.show()
```



7. Which features matter most?

```
if hasattr(best, "feature_importances_"):
    imp = pd.Series(best.feature_importances_, index=FEATURES).sort_values()
    imp.plot(kind="barh", color="steelblue", figsize=(6, 4))
    plt.title("Feature importance")
    plt.show()
```



8. Predict the weather

```
def predict_weather(precipitation, temp_max, temp_min, wind, month):
    row = pd.DataFrame([[precipitation, temp_max, temp_min, wind, month]],
                       columns=["precipitation", "temp_max", "temp_min", "wind", "month"])
    row["temp_range"] = row["temp_max"] - row["temp_min"]
    row["temp_avg"] = (row["temp_max"] + row["temp_min"]) / 2
    row["wet"] = (row["precipitation"] > 0).astype(int)
    return best.predict(row[FEATURES])[0]

samples = [
    ("Rain 10.9 mm, 10.6/2.8 C, wind 4.5, January", (10.9, 10.6, 2.8, 4.5, 1)),
    ("Dry, 28/14 C, wind 2.5, July",                (0.0, 28.0, 14.0, 2.5, 7)),
    ("Dry, 7.2/1.1 C, wind 1.8, December",          (0.0, 7.2, 1.1, 1.8, 12)),
    ("Light precipitation, 0.5/-3 C, January",      (5.0, 0.5, -3.0, 3.0, 1)),
]
```

```
for text, vals in samples:
    print(f"{text:<48} -> {predict_weather(*vals)}")
```

```
Rain 10.9 mm, 10.6/2.8 C, wind 4.5, January      -> rain
Dry, 28/14 C, wind 2.5, July                     -> sun
Dry, 7.2/1.1 C, wind 1.8, December               -> fog
Light precipitation, 0.5/-3 C, January           -> snow
```

9. Try your own values

Order: precipitation (mm), max temp (C), min temp (C), wind speed, month (1-12).

```
print(predict_weather(2.0, 12.0, 6.0, 3.5, 11))
```

```
rain
```

10. Download the trained model (optional)

```
import joblib
joblib.dump(best, "weather_model.joblib")
```